
Reinforcement Learning for Autonomous Navigation in a 2D Environment.

Eirik Varnes
Università di Bologna
`eirik.varnes@studio.unibo.it`

Abstract

Autonomous navigation is a key component of an autonomous underwater vehicle (AUV). In this project, we explore reinforcement learning (RL) as an alternative to traditional rule-based algorithms. The project focuses on the incremental development of both the environment and learning agent to gain a better understanding of how such systems can be designed.

1 Introduction

Autonomous navigation, particularly for AUVs operating in unmapped or dynamic conditions, poses significant challenges for traditional control methods. Reinforcement Learning provides a powerful framework for agents to learn effective navigation policies through environmental interaction, making it well-suited for such scenarios. This project investigates the application of RL to 2D autonomous navigation tasks, with a central focus on addressing partial observability, a realistic constraint where agents like AUVs rely on local sensor data without complete environmental knowledge.

The study follows an iterative development process. It begins with foundational explorations in a custom 2D grid world, where environmental complexity (e.g., obstacles) and agent perception (e.g., LiDAR-like sensors, memory buffers) were progressively implemented. This phase allowed for the evaluation of tabular Q-learning and Deep Q-Networks (DQNs), in these partially observable settings.

In later stages the project transitioned to a more complex 2D AUV simulation. This stage introduced continuous space, physics-based dynamics and continuous control. Adaptations included reward shaping and the implementation of the TD3 actor-critic algorithm for continuous actions.

This report details this developmental journey, outlining the methodology, experimental results, and discussion for each stage, followed by overall conclusions.

2 Foundational Approach

The scope of this project is primarily defined by the reinforcement learning (RL) concepts and principles covered within the Autonomous and Adaptive Systems course. The agents neural networks implementations, were developed using the PyTorch framework.

The custom environments, including the initial grid world and the subsequent Autonomous Underwater Vehicle (AUV) simulation, were designed to adhere to the standards and interface of the original OpenAI Gym (now Gymnasium) environment, facilitating standard agent-environment interaction.

The thematic elements of the project, particularly those related to AUV navigation, sensor modeling (such as LiDAR and sonar), and environmental physics, draw upon knowledge acquired from previous internships and the personal interest of the author in these specific application domains.

3 Method

The methodology for this project involved several stages of development, aiming to explore and implement reinforcement learning (RL) solutions for autonomous navigation tasks. A primary consideration throughout was the use of partial observations, reflecting realistic conditions where an agent, such as an autonomous underwater vehicle (AUV), must operate without complete prior knowledge of its surroundings. This section details these stages, beginning with foundational work in a custom grid world environment, followed by a transition to a more complex AUV simulation.

3.1 Stage 1: Foundational Exploration in a Grid World Environment

The initial stage focused on establishing core RL navigation principles in a controlled 2D grid world. The approach was iterative: starting with a basic setup and progressively introducing complexities to the environment and agent perception to investigate learning with limited sensory input.

3.1.1 Initial Setup: Basic Grid Navigation with Q-learning

A custom environment, GridEnv, was implemented adhering to the OpenAI Gym interface. This environment is a configurable 2D grid where an agent aims to navigate from a start to a target position. Actions were discrete (up, down, left, right). The agent's initial observation was its 2D position relative to the goal. A simple reward function was used: +1.0 for reaching the goal and -1.0 for each step taken, encouraging efficient paths.

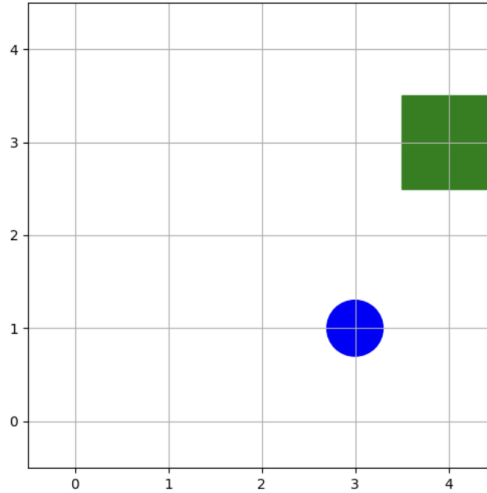


Figure 1: A simple 5x5 grid world environment with the agent (blue circle) and the goal (green square), without obstacles.

A tabular Q-learning agent was first implemented. Q-learning is a model-free algorithm that learns an optimal action-value function ($Q^*(s, a)$) by iteratively updating Q-table entries using the Bellman equation: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$. This approach was chosen for its simplicity in understanding fundamental RL mechanics in a constrained state space. While successful in small grids, its limitations became apparent in larger grids where it achieved 98.1% success but struggled with scalability compared to a Deep Q-Network (DQN). The need for function approximation for larger state spaces motivated the subsequent use of DQNs.

3.1.2 Addressing Unseen Obstacles: The Need for Enhanced Perception

Obstacles were then introduced into the grid world, which could be placed randomly or statically. If the agent attempted to move into an obstacle, its position remained unchanged. Initially, the agent (now a DQN, better suited for potentially larger state spaces) had no direct perception of these obstacles, relying only on its relative position to the goal. This led to significant navigation challenges and unstable training if direct paths were blocked, as the agent could not distinguish a blocked action from a successful one leading to no apparent change in relative goal position. This highlighted a critical limitation of the current state representation.

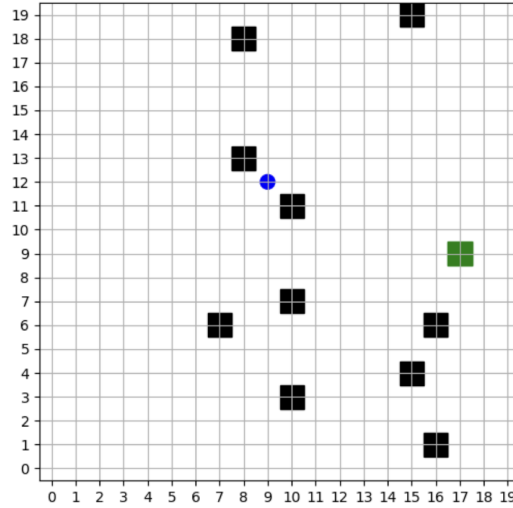


Figure 2: Example of a 20x20 grid world environment with obstacles (black squares), agent (blue circle), and goal (green square).

3.1.3 Introducing Local Perception: LiDAR-like Sensors

To overcome the inability to perceive obstacles, the agent's observation was augmented with LiDAR-like sensors. These sensors provided the DQN with four distance readings to the nearest obstacle or boundary in cardinal directions, up to a defined range. This addition transformed the problem by giving the agent explicit local environmental awareness. The tabular Q-learning approach was clearly impractical for this higher-dimensional state space, reinforcing the necessity of the DQN's function approximation capabilities. With LiDAR data, the DQN's performance improved substantially to a 75.8% success rate, demonstrating its ability to learn obstacle avoidance. However, complex maze-like configurations revealed limitations, as the agent could still converge to sub-optimal paths or local minimums.

3.1.4 Enhancing State Disambiguation: History Buffer

To address the remaining challenges, particularly in maze-like scenarios where current perception alone is insufficient, a short-term memory mechanism was implemented via a history buffer. This augmented the DQN's state by including a sequence of recent observations (relative position and LiDAR readings). The rationale was that temporal context could help the agent disambiguate states and recognize patterns like oscillations or previously explored dead ends. This enhancement further improved the success rate to 93%, with remaining failures often due to physically unreachable goals.

3.2 Stage 2: Navigation in a Continuous 2D AUV Environment

Building on the grid world insights, the project transitioned to a more realistic 2D continuous-space simulation of an Autonomous Underwater Vehicle (AUV). This stage aimed to tackle challenges like continuous movement, larger state spaces, more complex sensor models, and the influence of physics.

3.2.1 AUV Environment Design

A new environment, AUVEnv, was developed, featuring a 2D continuous operational space defined by a `grid_size` (e.g., 200x200 units) and `resolution` (e.g., 0.1 units per internal grid cell for obstacle representation). This allowed for floating-point positions and orientations for the AUV. Obstacles were represented on an occupancy grid, with options for both statically defined layouts (via `build_maps`) and randomly generated maps using a cellular automaton approach (via `build_random_maps`). In each episode, the AUV and its goal dock(s) were spawned at random locations within obstacle-free areas of the map.

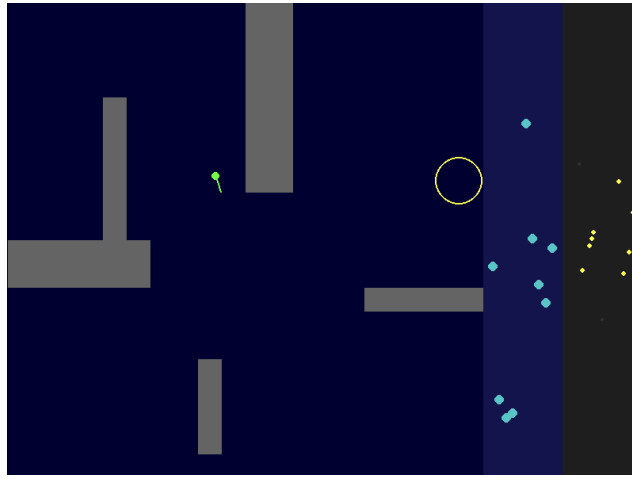


Figure 3: Snapshot of the static AUV environment. The AUV (green), goal dock (yellow circle), and obstacles (grey) are shown. Side panels visualize sonar hits.

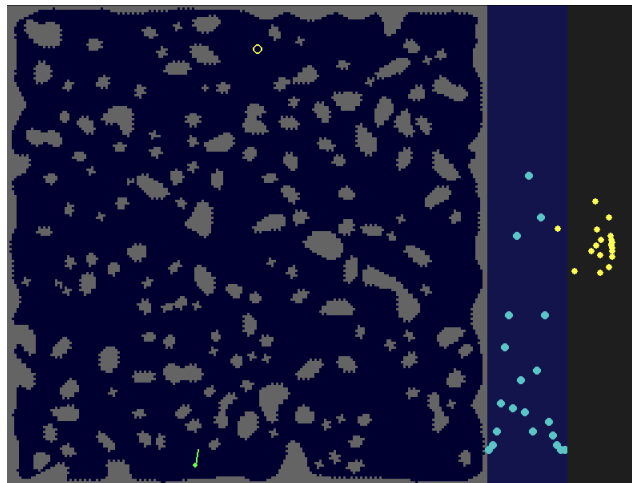


Figure 4: Snapshot of a randomly generated AUV environment, showing a more cluttered navigation challenge with similar sonar visualizations.

3.2.2 Physics-Based Dynamics Model

A key development for this stage was the introduction of optional physics-based movement, governed by the `use_physics` configuration flag. When enabled, the AUV's motion is no longer purely kinematic. Instead, its linear velocity ($\mathbf{v}$) and position ($\mathbf{p}$) are updated based on forces, and its heading (θ) is updated based on torque. The net force acting on the AUV considers applied thrust and drag: $\mathbf{F}_{net} = \mathbf{F}_{thrust} + \mathbf{F}_{drag}$. Thrust is applied along the AUV's current heading. Drag is modeled as a linear drag force: $\mathbf{F}_{drag} = -c_d \mathbf{v}$, where c_d is the `drag_coef`. Acceleration is then $\mathbf{a} = \mathbf{F}_{net}/m$, where m is the mass. The state is updated using Euler integration with a time step Δt (e.g., 0.1 seconds, defined by `dt`):

$$\mathbf{v}_{t+1} = \mathbf{v}_t + \mathbf{a}_t \Delta t$$

$$\mathbf{p}_{t+1} = \mathbf{p}_t + \mathbf{v}_{t+1} \Delta t$$

$$\theta_{t+1} = \theta_t + (\text{torque}_t/m) \Delta t$$

(Note: The angular update $\theta_{t+1} = \theta_t + (\text{torque}_t/m) \Delta t$ implies torque causes angular acceleration scaled by mass, which acts as a proxy for moment of inertia here). The environment also supports configurable ocean currents, which can add an additional velocity component to the AUV's motion. This physics model introduces inertia and makes the control task significantly more challenging.

3.2.3 AUV Action Space Design

The AUVEnv supports two action modes, selectable via the `use_discrete_actions` configuration:

- **Discrete Actions:** When enabled, the agent selects from a predefined set of maneuvers. The default discrete action set (`DEFAULT_DISCRETE_ACTIONS`) includes five actions:
 1. Forward (e.g., `thrust=0.3, torque=0.0`)
 2. Forward + Turn Left (e.g., `thrust=0.3, torque=0.3 rad/s`)
 3. Forward + Turn Right (e.g., `thrust=0.3, torque=-0.3 rad/s`)
 4. Turn Left (In-place) (e.g., `thrust=0.0, torque=0.3 rad/s`)
 5. Turn Right (In-place) (e.g., `thrust=0.0, torque=-0.3 rad/s`)

These values represent target linear velocity and angular velocity components.

- **Continuous Actions:** When discrete actions are disabled, the agent outputs a 2D action vector: `[thrust, torque]`. The thrust component is typically scaled to an operational range (e.g., `[-0.4, 0.4]` m/s, based on `DEFAULT_MAX_THRUST`), and the torque is scaled to an operational range (e.g., `[-0.4, 0.4]` rad/s, based on `DEFAULT_MAX_TORQUE`). The neural network policy for continuous control typically outputs values in `[-1, 1]`, which are then scaled to these operational ranges.

3.2.4 AUV Observation Space Design

The AUV's observation vector was designed to provide comprehensive local information necessary for navigation with partial observability:

- **Sonar Sensing:** A 360-degree sonar sensor (`SonarSensor` class) provides multiple (e.g., 20, defined by `n_beams`) normalized distance readings to obstacles or boundaries around the AUV, up to a maximum range (e.g., 10 units). A binary hit mask indicates which beams detected an obstacle.
- **Goal Information:** For each goal dock, the observation includes its normalized distance from the AUV and the sine and cosine of the relative angle to the dock, calculated with respect to the AUV's current heading.
- **Action History (Optional):** If `use_history` is enabled, a sequence of the agent's most recent actions is concatenated to the observation, processed by the `HistoryBuffer`.

The complete observation vector is a concatenation of these elements.

3.2.5 AUV Reward Function Design

The sparse reward function from the grid world (+1 for goal, -1 per step) proved insufficient for the larger and more complex AUVEnv. In such a vast state space, reaching the goal through purely random exploration to receive the sparse positive signal is highly improbable. This necessitated the careful design of a shaped reward function to provide denser feedback and guide the learning process more effectively.

Arriving at the final set of reward components and their specific coefficients involved considerable experimentation and iteration. This included an analysis of typical reward magnitudes to ensure a proper balance; for instance, the main docking reward was set significantly higher than potential accumulated negative rewards from step costs over an average episode, ensuring that reaching the goal remained the primary driver for the agent.

The final shaped reward function implemented in the `AUVEnv.step` method comprised several components. The example values and formulas shown in Table 1 correspond to the default settings for the discrete action mode.

Table 1: Components of the Shaped Reward Function for the AUV Environment (Discrete Action Mode Examples)

Component	Description	Formula
Docking Reward	Large positive reward upon successfully reaching a dock.	+500.0
Step Cost	Small negative reward for each time step taken to encourage efficiency.	-1.0
Wall Proximity Penalty	Penalty if the AUV’s minimum sonar reading ($R_{\min}$, normalized) is less than a threshold (T_{wall} , e.g., 1.0 unit). Coefficient C_{wall} is, e.g., 20.0.	$-C_{\text{wall}} \times (1 - \frac{R_{\min}}{T_{\text{wall}}})$
Progress Reward	Positive reward proportional to the reduction in distance to the goal dock ($\Delta d = \text{prev_dist} - \text{curr_dist}$). Coefficient C_{prog} is, e.g., 3.0.	$+C_{\text{prog}} \times \Delta d$
Turn Penalty	Penalty proportional to the magnitude of the applied torque ($ \text{torque} $). Coefficient C_{turn} is, e.g., 30.0.	$-C_{\text{turn}} \times \text{torque} $

This combination of reward components was found to be crucial for enabling the agents to learn meaningful navigation policies in the AUV environment. It is important to note that the specific coefficients for these components (e.g., C_{wall} , C_{prog} , C_{turn}) and the primary rewards like docking reward and step cost, have counterparts for the continuous action mode (e.g., `DEFAULT_CONTINUOUS_WALL_PENALTY_COEFF`, etc.), which were tuned to balance the scale of continuous action values against the reward signals. The overall process of defining and tuning these reward terms was iterative, aiming to encourage desired behaviors while preventing agents from exploiting specific terms in unintended ways.

3.2.6 Agent Implementations and Rationale in AUV Environment

An initial Deep Q-Network (DQN) agent, employing a Multi-Layer Perceptron (MLP) for function approximation, experience replay, and target networks, was first applied to the AUVEnv. This agent operated with the *discrete action set* and the shaped reward signal. In kinematic settings (i.e., without physics-based dynamics enabled), this DQN configuration demonstrated reasonable performance in learning navigation tasks.

Given that the underlying control problem for an AUV is continuous, especially when considering physics, using a discrete action space (where the agent chooses from a small, predefined set of maneuvers) represents a simplification. To explore finer and potentially more robust control, the environment was configured to support a *continuous action space*. This allows the agent to select precise values for its controls, such as the exact amount of thrust and torque, offering more nuanced command over its movement compared to fixed maneuvers. Standard Deep Q-Network (DQN) algorithms are primarily designed for discrete action spaces, as they learn to select the action with the highest Q-value from a finite list. Therefore, a different algorithmic approach was necessary for the continuous action domain.

The Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm was chosen and implemented. TD3 is an **actor-critic method**, meaning it uses two distinct neural networks: an 'actor' network that learns the policy (i.e., decides which action to take in a given state), and one or more 'critic' networks that learn an action-value function to evaluate the actions chosen by the actor. Specifically, TD3 is based on a **deterministic policy gradient**, where the actor network directly outputs the specific continuous action to take for a given state, rather than a probability distribution over possible actions. TD3 is designed for stable continuous control and incorporates several key features to achieve this. These include the use of two 'twin' critic networks, where the minimum of their Q-value predictions is used for calculating the learning targets, a technique which helps to reduce the common problem of overestimating Q-values. Furthermore, the policy (actor) and the target networks (slowly-updated copies of the main networks used to stabilize learning) are updated less frequently than the critic networks; this 'delayed' update strategy provides the actor with more stable learning targets from the critics. TD3 also employs target policy smoothing, where noise is added to the target action, which helps prevent the policy from exploiting peaks in the action-value function. These features collectively contribute to mitigating Q-value overestimation and improving overall learning stability.

Before applying TD3 to the physics-based environment, its effectiveness was first confirmed in the no physics environment using continuous actions. In these initial tests, TD3 demonstrated performance comparable to, and often exceeding, that of the discrete-action DQN agent. The decision to employ TD3 for the physics-enabled tasks was therefore based on its demonstrated effectiveness in the kinematic AUV setup with continuous actions, its inherent design for continuous control (which was hypothesized to be more adept at managing the complex dynamics introduced by the physics engine), and its established stability-enhancing features, especially where the previously tested discrete-action approaches had struggled.

4 Experiments

This section describes the setup for the experiments that were done to test the learning agents and environments from the Method section. It presents the results from each stage of development. This will show how well the different approaches worked and what happened when things like the agent’s sensors, memory, or the environment’s difficulty were changed.

4.1 Stage 1: Experiments in the Grid World Environment

The first stage of experiments was about setting up basic navigation and seeing how things like partial information, obstacles, and better sensors affected learning in the grid world.

4.1.1 Results of Iterative Development in Grid World

Phase 1.1: Basic Navigation (Q-learning vs. DQN) In the basic 5x5 grid world with no obstacles (shown in Figure 1), the Q-learning agent learned well. The learning curve in Figure 5 shows that the total reward per episode quickly reached a steady high value (near 0). This means the agent learned to find the goal efficiently.

For a larger 20x20 grid with no obstacles, the Q-learning agent had a 98.1% success rate. The learning curve in Figure 6 shows that it learned, with the 1000-step moving average of rewards settling around -10. It took about 20,000 to 30,000 episodes for this to happen.

A DQN agent trained on the same 20x20 grid without obstacles (Figure 7) achieved a 100% success rate and learned faster. Its 10-step moving average for rewards reached near-zero values, and relative low variance, in under 1000 episodes. This showed that DQN was better for larger state spaces even in this simple task.

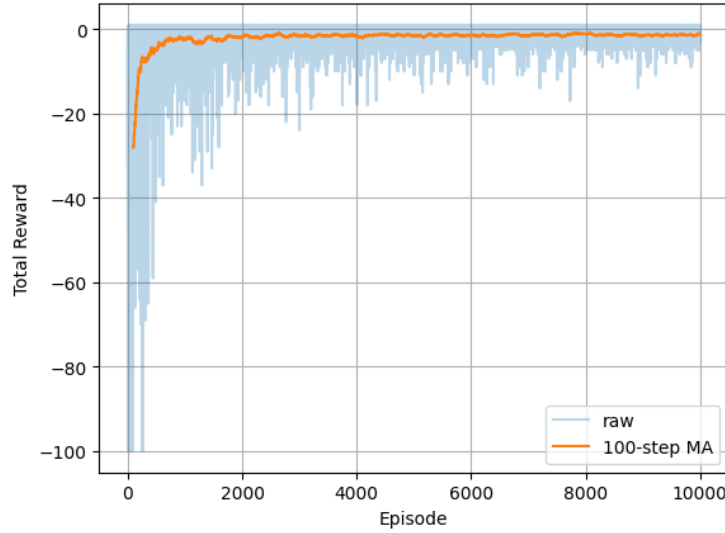


Figure 5: Q-Learning training progress in the 5x5 basic grid world. Total reward per episode (raw) and a 100-step moving average (MA).

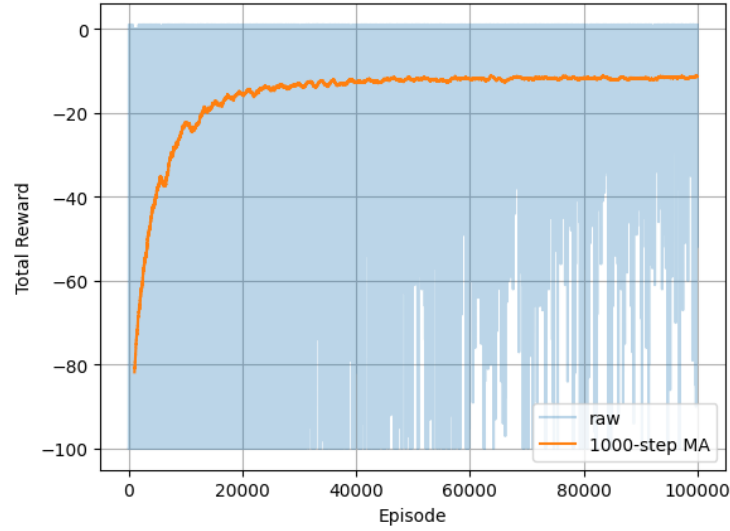


Figure 6: Q-Learning training progress in the 20x20 basic grid world. Total reward per episode (raw) and a 1000-step moving average (MA).

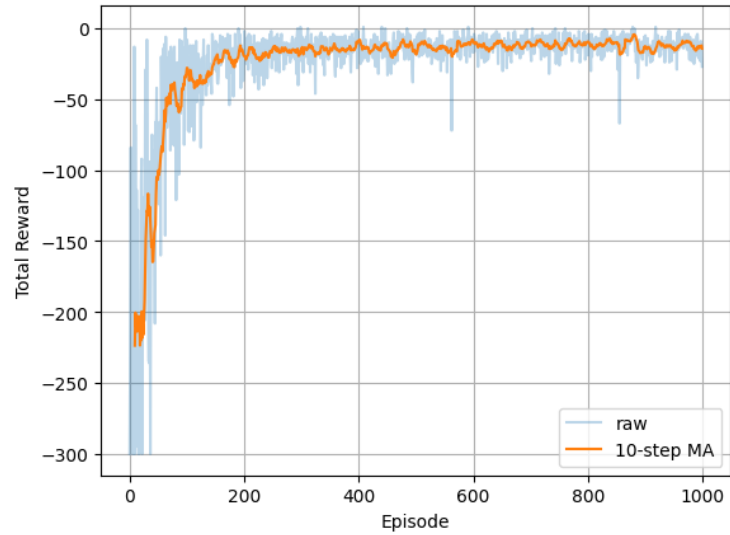


Figure 7: DQN training progress in the 20x20 basic grid world. Total reward per episode (raw) and a 10-step moving average (MA).

Phase 1.2: Obstacles without Agent Perception (DQN) When obstacles were added to the 20x20 grid world (an example is in Figure 2), the DQN agent that only knew its relative position to the goal (no LiDAR) still had a success rate of 59.7% in 100 test runs. Successful paths took an average of 12.3 steps. However, learning was difficult. Figure 8 shows that the total reward per episode varied a lot. The 20-step moving average shows some learning, but it stayed low (around -25 to -50) and was not stable. This means that even though the agent found the goal many times (maybe when paths were clear or by chance), it struggled to learn a good way to avoid obstacles because it could not see them directly. It essentially had no way of knowing why sometimes actions did not result in the expected change in state.

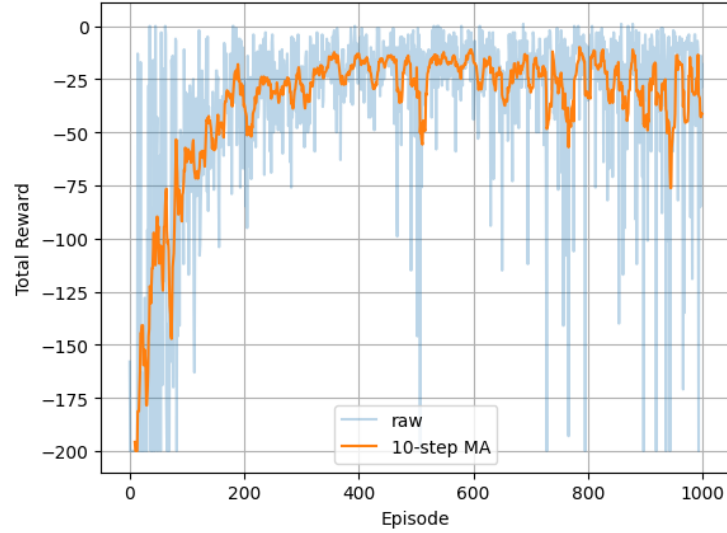


Figure 8: DQN training progress in the 20x20 grid world with obstacles but no LiDAR sensors. Total reward per episode (raw) and a 20-step moving average (MA).

Phase 1.3: Perception with LiDAR Sensors (DQN) Adding 4-directional LiDAR sensors gave the DQN agent local information about what was around it. This made a big difference in the 20x20 grid with obstacles. The success rate went up to 82.3%, and successful runs averaged 12.8 steps. The learning curve in Figure 9 shows this improvement: the 10-step moving average of total rewards increased more steadily and became stable at a higher level than without LiDAR. This shows the agent used the LiDAR information to learn to avoid obstacles. The average steps being slightly higher than the no-LiDAR successful runs (12.8 vs 12.3) might mean the agent was now taking careful paths around obstacles.

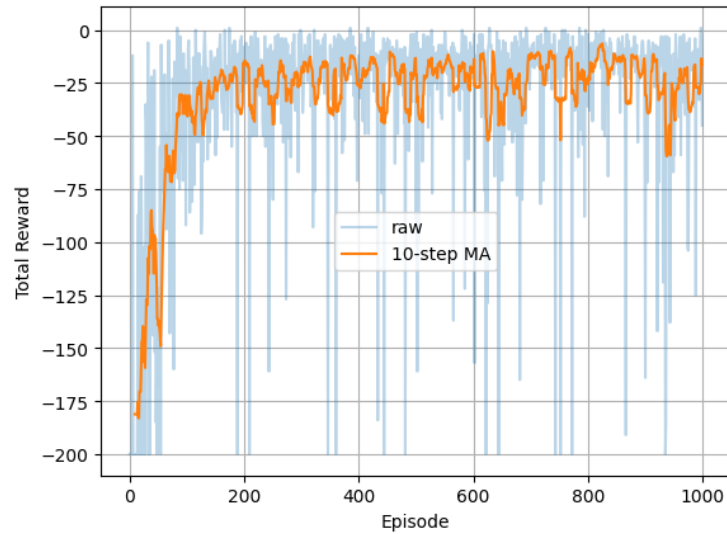


Figure 9: DQN training progress in the 20x20 grid world with obstacles and LiDAR sensors. Total reward per episode (raw) and a 10-step moving average (MA).

Phase 1.4: Memory with History Buffer (DQN) Adding a history of recent observations (including LiDAR data) to the DQN agent’s state helped even more in the complex 20x20 grid with obstacles. The success rate reached 94.3%. The learning curve in Figure 10 shows that the 10-step moving average of total rewards became generally higher and more stable than with only LiDAR. Successful episodes now took an average of 18.4 steps. This increase in average steps, along with a higher success rate, suggests the agent with memory could solve more difficult mazes that needed longer paths, which the other agents might have failed on. The history buffer likely helped the agent understand its situation better and avoid repeating bad moves. Failures at this point were mostly when the goal was completely blocked by obstacles.

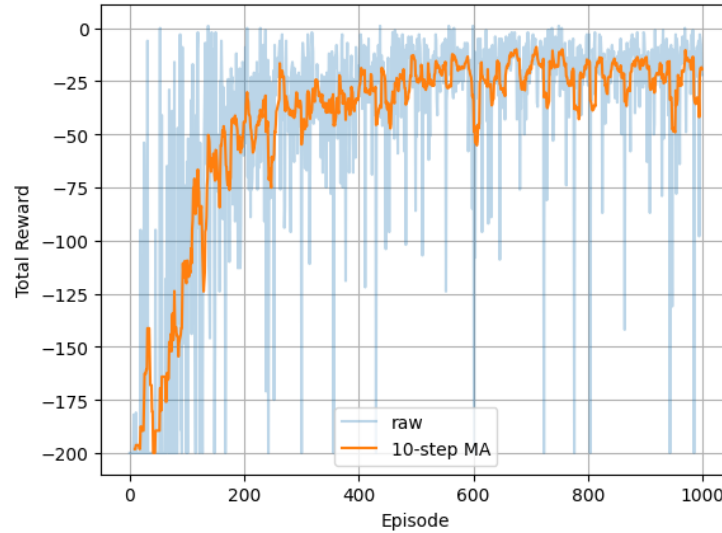


Figure 10: DQN training progress in the 20x20 grid world with obstacles, LiDAR, and history buffer. Total reward per episode (raw) and a 10-step moving average (MA).

4.2 Stage 2: Experiments in the AUV Environment

4.2.1 DQN Performance with Discrete Actions (No Physics)

The first DQN agent, using discrete actions and the shaped reward function in the AUV environment (without physics), showed it could learn the basics. In the **static environment**, the DQN agent had a success rate of 97.6% and took an average of 29.9 steps when successful. The learning curve for this (Figure 11) shows the 50-step MA of total rewards mostly staying near zero, meaning the agent learned to reach the goal without too many penalties. In **randomly generated environments**, the DQN's success rate was 86.0%, with successful tries averaging 34.3 steps. Its learning curve (Figure 12) also shows learning, with the 50-step MA of rewards reaching positive values, but it varied more than in the static case. This shows it was harder to learn for many different map layouts. It was also tested on a new randomly generated environment, where the success rate was 91.4 %.

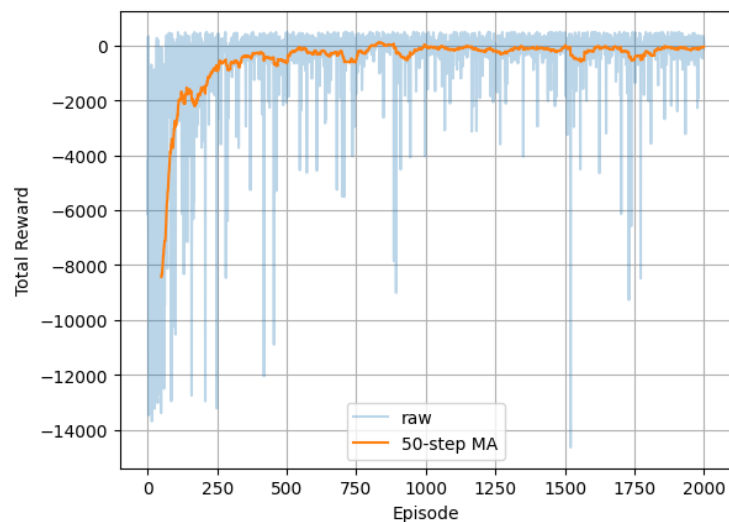


Figure 11: DQN training progress in the static AUV environment (discrete actions, no physics). Total reward per episode (raw) and 50-step MA.

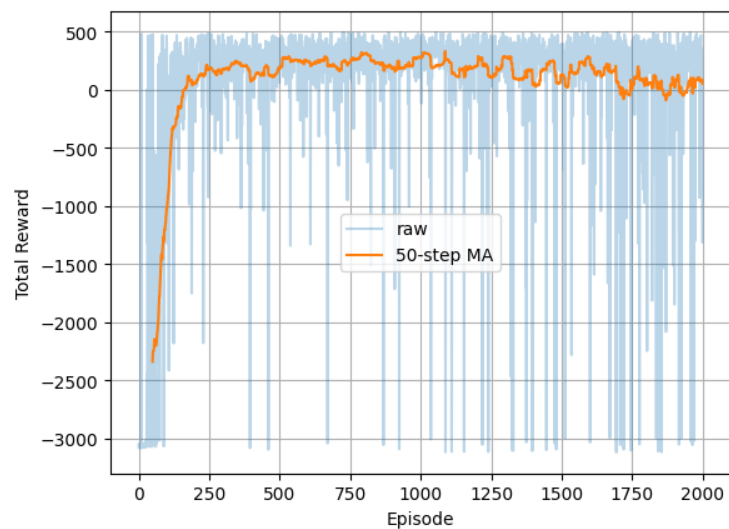


Figure 12: DQN training progress in randomly generated AUV environments (discrete actions, no physics). Total reward per episode (raw) and 50-step MA.

4.2.2 TD3 Agent Performance (Continuous Actions, No Physics)

A TD3 agent, which uses continuous actions, was then tested. **Without physics**, TD3 performed better than DQN. In the **static AUV environment**, it had a 99.0% success rate, averaging at 36.1 steps when successful. Its learning curve (Figure 13) shows it learned quickly, with the 50-step MA reaching very high total rewards. In **randomly generated environments without physics**, TD3 also did very well, with a 91.0% success rate and averaging 33.1 steps. Its learning curve (Figure 14) also shows strong learning and high rewards. This shows that a continuous action agent was very effective for the AUV task when movement was simple (kinematic).

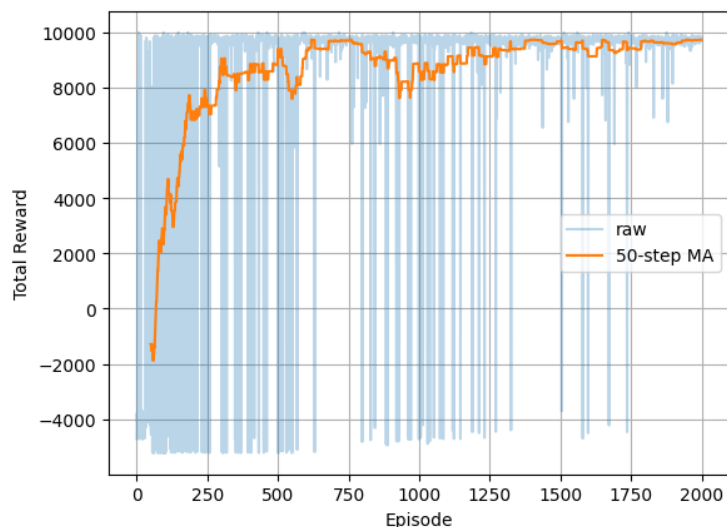


Figure 13: TD3 training progress in the static AUV environment (continuous actions, no physics). Total reward per episode (raw) and 50-step MA.

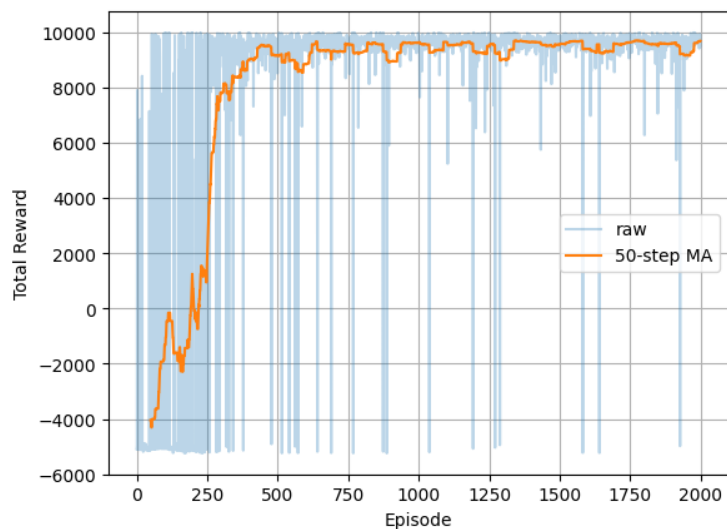


Figure 14: TD3 training progress in randomly generated AUV environments (continuous actions, no physics). Total reward per episode (raw) and 50-step MA.

4.2.3 TD3 With Physics

The TD3 agent found this task much harder. In the **static environment with physics**, the success rate fell to 25.0%, and successful tries took a long time (average 131.1 steps). The learning curve (Figure 15) shows this difficulty, with very low total rewards, meaning many penalties and trouble reaching the goal. In **randomly generated environments with physics**, performance was even worse, with a success rate of only 0.4% and an average of 1.0 steps for the few successful tries. The learning curve (Figure 16) also shows very low rewards, with only a few spikes showing successful episodes.

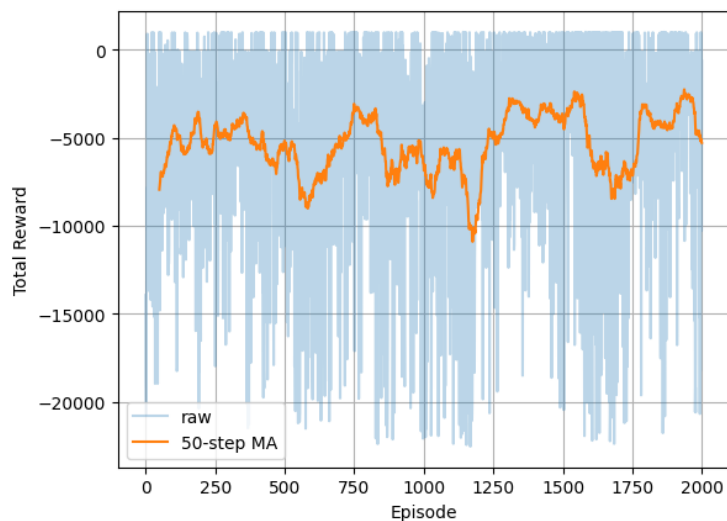


Figure 15: TD3 training progress in the static AUV environment with physics enabled (continuous actions). Total reward per episode (raw) and 50-step MA.

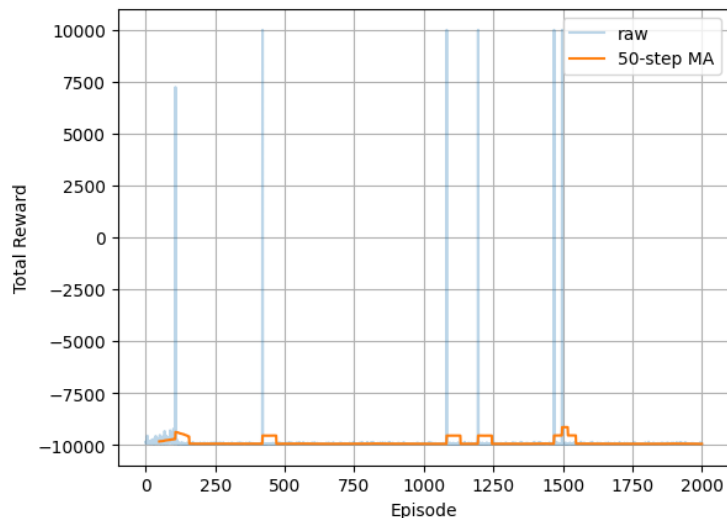


Figure 16: TD3 training progress in randomly generated AUV environments with physics enabled (continuous actions). Total reward per episode (raw) and 50-step MA.

5 Discussion

This project explored the application of reinforcement learning (RL) to autonomous navigation tasks, progressing from a foundational grid world to a more complex 2D environment. The experiments conducted at each stage provide several insights into the capabilities and challenges of different RL agents and environment designs.

5.1 Interpretation of Key Findings

State Representation and Partial Observability: A central theme was the agent’s reliance on partial observations. In the initial grid world, the experiments demonstrated that a basic state representation (relative goal position) was insufficient when unperceivable obstacles were introduced. The DQN agent’s performance, which showed unstable learning and a **59.7%** success rate when obstacles were present but not sensed, significantly improved with the addition of LiDAR-like sensors (to **82.3%** success, and further with a history buffer for short-term memory (to **94.3%** success. This underscores the importance of sufficient sensory input and memory for enabling effective navigation in partially observable environments.

Algorithm Scalability and Suitability: The transition from tabular Q-learning to Deep Q-Networks (DQN) in the grid world highlighted the scalability limitations of tabular methods. While Q-learning was effective, DQN demonstrated superior performance (**100%** success and faster convergence in under 1000 episodes) compared to Q-learning (**98.1%** success and slower convergence requiring 20,000 to 30,000 episodes) in the larger 20x20 grids, even before sensors were added. This justified the use of DQNs and other neural network-based agents for the significantly larger state spaces encountered in both the sensor-augmented grid world and the AUV simulation.

In the AUV environment without physics, the TD3 agent, designed for continuous action spaces, generally outperformed the DQN with discrete actions. For instance, in the **static AUV environment without physics**, TD3 achieved a **99.0%** success rate compared to DQN’s **97.6%**. In **randomly generated AUV environments without physics**, TD3 achieved **91.0%** success compared to DQN’s **86.0%**. This suggests that for environments with continuous underlying dynamics, agents capable of outputting continuous actions can learn more effective and nuanced control policies.

Impact of Environmental Complexity and Physics: The introduction of physics-based dynamics in the AUV environment (Stage 2) posed a considerable challenge. The TD3 agent saw a dramatic drop in performance when physics was enabled. For example, TD3’s success rate in the **static AUV environment dropped from 99.0% (no physics) to 25.0% with physics**. In **randomly generated environments, this drop was even more stark, from 91.0% (no physics) to a mere 0.4% with physics**. The learning curves for physics-enabled scenarios consistently showed very low rewards and instability. This indicates that the increased complexity from inertia, drag, and more nuanced agent-environment interactions requires either more sophisticated agent architectures, more refined state representations, or more advanced reward shaping and exploration strategies than those implemented. The note about an Enhanced DQN (with Dueling architecture and Prioritized Experience Replay) not showing noteworthy improvement during development further emphasizes this difficulty.

Reward Shaping: The necessity of reward shaping became important in the AUV environment. The simple sparse reward (e.g., +1 for goal, small penalty per step) that worked in the grid world was insufficient to guide learning in the AUV state space. The implementation of a shaped reward function, incorporating terms for progress towards the goal, penalties for wall proximity, and other behaviors like excessive turning, along with a step cost—was key for enabling both DQN and TD3 agents to learn meaningful behaviors in the AUV environment. This highlights that as task complexity increases, providing denser and more informative reward signals is essential for effective learning.

5.2 Limitations of the Current Work

Despite the insights gained, this project has several limitations:

- **Performance in Physics-Based AUV Environment:** The most significant limitation is the low success rate of the TD3 agent in the AUV environment when physics were enabled (e.g., **25.0%** in static, **0.4%** in random environments). The current agent architectures and reward structures were not sufficient to overcome the complexities introduced by these dynamics.
- **Sensor and Physics Realism:** While the sonar/LiDAR model and physics in the AUV environment add realism compared to the grid world, they are still simplifications. Real-world sonar is affected by more complex noise and environmental factors.
- **Exploration Strategies:** The project primarily relied on ϵ -greedy exploration for DQN and inherent noise for TD3 during training. More sophisticated exploration techniques might be needed for the challenging AUV environments, especially with physics.
- **Hyperparameter Tuning:** RL algorithms are known to be sensitive to hyperparameter settings. While some tuning was performed, more extensive tuning could potentially yield improved performance for all agents.
- **Limited Agent Architectures Explored:** While Q-learning, DQN, and TD3 (along with considerations for Dueling DQN and PER) were covered, other advanced RL algorithms (e.g., Soft Actor-Critic (SAC), Proximal Policy Optimization (PPO)) that might be suited for these tasks were not implemented.

5.3 Future Work

Looking ahead, there are a few main ways to build on this project:

First, we need to **make the AUV agent much better at navigating when real-world physics are involved**. This means trying newer AI learning methods, designing smarter agents, finding better ways to reward good actions, and helping the agent explore its surroundings more effectively.

Second, we can **make the AUV’s simulated practice world more like the actual ocean**. This involves adding more true-to-life water effects, making the sensors act more realistically, and testing features like simulated ocean currents.

Additionally, we could explore **combining our learning AI with proven engineering techniques for robot control and navigation**. For instance, using methods like SLAM (to help the AUV build maps and know where it is) or Kalman filters (to get clearer information from sensors) could team up with the AI agent, potentially making its navigation much more robust and efficient.

6 Conclusion

This project successfully demonstrated the iterative development and application of reinforcement learning agents for autonomous navigation, progressing from basic grid worlds to a more complex 2D AUV simulation. Key findings underscored the critical role of adequate sensory input and memory in partially observable environments, with DQN performance in grid worlds improving from 59.7% to 94.3% success with the addition of sensors and history. The scalability advantage of neural network agents like DQN over Q-learning in larger state spaces was confirmed. Furthermore, in kinematic AUV simulations, the continuous control capabilities of TD3 yielded superior results (e.g., 99.0% success in static environments without physics) compared to discrete-action DQN (97.6%).

However, the introduction of physics-based dynamics in the AUV environment presented a substantial challenge, with TD3 agent performance dropping significantly (e.g., to 25.0% in static and 0.4% in random environments with physics), highlighting a primary limitation. While reward shaping proved essential for learning in the AUV context, generalization to unseen environments, especially under complex dynamics, remains an area for improvement.

The insights gained lay the groundwork for future explorations into more sophisticated agent architectures, advanced reward engineering, and enhanced simulation realism, aiming to bridge the gap towards robust autonomous navigation in complex, dynamic systems.